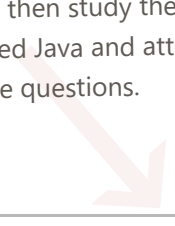


Compiling and Running Packages

Compile source into package directories, then run the fully qualified class from the correct classpath root.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.



The whole process at a glance

Compile source into package directories, then run the fully qualified class from the correct classpath root.

Think of a package as a class address. The package, filename, source root and compiled root have distinct jobs.

1

Declare the address

```
package pkg; public class Tester
```

The class identity is pkg.Tester. Put the declaration before imports and the class.

**2**

Place the source

```
src/pkg/Tester.java
```

src is the source root. pkg mirrors the package name. A public Tester class needs the filename Tester.java.

**3**

Compile the project

```
javac -d classes -sourcepath src src/pkg/Tester.java
```

From the project directory, the compiler finds dependencies under src and writes classes/pkg/Tester.class.



Run the entry point

```
java -classpath classes pkg.Tester
```

classes is the root above pkg. Use a dotted class name, without .class, and invoke its public static main method.

The concept, plain language and vocabulary

Compilation translates Java source into class files. Running a packaged class combines a classpath root with a fully qualified class name and invokes its main method.

```
javac  
reads src/pkg/Tester.java
```



```
-d classes  
writes classes/pkg/Tester.class
```



```
java resolves classes +  
pkg.Tester
```

Runtime lookup combines the classpath root with the fully qualified binary name, then invokes main.

TERM	MEANING IN THIS CHAPTER
javac	Java compiler: source .java to compiled .class.
-sourcepath src	Where to find source dependencies.
-d classes	Where to write compiled package directories.
-classpath classes	Where Java begins lookup of compiled classes.
Fully qualified name	pkg.Tester, not a slash path and not a filename.

```
javac  
reads src/pkg/Tester.java
```



```
-d classes  
writes classes/pkg/Tester.class
```



```
java resolves classes +  
pkg.Tester
```

Runtime lookup combines the classpath root with the fully qualified binary name, then invokes main.

Source root: src/

Output / classpath root: classes/

Example

```
src/  
  pkg/  
    A.java  
    B.java  
    Tester.java  
  app/  
    Main.java
```

Java

```
classes/  
  pkg/  
    A.class  
    B.class  
    Tester.class  
  app/  
    Main.class
```

Package identity is below each root. src and classes are roots, not parts of the package name.

Guided walkthrough: predict the next major step

Follow the class from source to execution

Declare the address

1

Declare the address

```
package pkg; public class Tester
```

The class identity is pkg.Tester. Put the declaration before imports and the class.



The class identity is pkg.Tester. Put the declaration before imports and the class.

Place the source

2

Place the source

```
src/pkg/Tester.java
```

src is the source root. pkg mirrors the package name. A public Tester class needs the filename Tester.java.



src is the source root. pkg mirrors the package name. A public Tester class needs the filename Tester.java.

Compile the project

3

Compile the project

```
javac -d classes -sourcepath src src/pkg/Tester.java
```

From the project directory, the compiler finds dependencies under src and writes classes/pkg/Tester.class.



From the project directory, the compiler finds dependencies under src and writes classes/pkg/Tester.class.

Run the entry point

4

Run the entry point

```
java -classpath classes pkg.Tester
```

classes is the root above pkg. Use a dotted class name, without .class, and invoke its public static main method.

classes is the root above pkg. Use a dotted class name, without .class, and invoke its public static main method.

▼ Full trace table - select a row to jump to its scene

STEP	STAGE	REPRESENTATION	REASON
1	Declare the address	package pkg; public class Tester	The class identity is pkg.Tester. Put the declaration before imports and the class.
2	Place the source	src/pkg/Tester.java	src is the source root. pkg mirrors the package name. A public Tester class needs the filename Tester.java.
3	Compile the project	javac -d classes -sourcepath src src/pkg/Tester.java	From the project directory, the compiler finds dependencies under src and writes classes/pkg/Tester.class.

STEP	STAGE	REPRESENTATION	REASON
4	Run the entry point	java -classpath classes pkg.Tester	classes is the root above pkg. Use a dotted class name, without .class, and invoke its public static main method.

The trace at a glance

STAGE	REPRESENTATION	WHAT CHANGES
Declare the address	package pkg; public class Tester	The class identity is pkg.Tester. Put the declaration before imports and the class.
Place the source	src/pkg/Tester.java	src is the source root. pkg mirrors the package name. A public Tester class needs the filename Tester.java.
Compile the project	javac -d classes -sourcepath src src/pkg/Tester.java	From the project directory, the compiler finds dependencies under src and writes classes/pkg/Tester.class.
Run the entry point	java -classpath classes pkg.Tester	classes is the root above pkg. Use a dotted class name, without .class, and invoke its public static main method.

Java packages

Group related classes together

A package is a named group of related classes organised in a matching folder structure. Packages improve maintainability and prevent class-name conflicts.

Built-in packages

Provided by Java, such as `java.lang`, `java.util`, and `java.io`.

User-defined packages

Created by programmers to organise their own related classes.

Common built-in examples

PACKAGE	PURPOSE	EXAMPLE
<code>java.lang</code>	Fundamental Java classes; imported automatically	<code>String</code> , <code>Math</code>
<code>java.util</code>	Utilities and collections	<code>Scanner</code> , <code>Random</code> , <code>Arrays</code>
<code>java.io</code>	File input and output	<code>PrintWriter</code> , <code>File</code>

A source file with no **package** statement belongs to the unnamed *default package*. Classes in a named package cannot import classes from the default package, so every file in this chapter declares a package. (The earlier array and file examples are single files in the default package, which is fine for a one-file program.)

The required folder hierarchy

```
project/
├── src/
│   ├── pkg/
│   │   ├── A.java
│   │   ├── B.java
│   │   └── Tester.java
│   └── app/
│       └── Main.java
└── classes/ ← created by javac -d
    ├── pkg/
    │   ├── A.class
    │   ├── B.class
    │   └── Tester.class
    └── app/
        └── Main.class
```

The `pkg` folders under `src` and `classes` mirror each other. The package name is the folder path with dots instead of slashes.

Inside every packaged source file

A offers a method, B a second helper, and `Tester` is the class you run because it alone contains `main`.

```
src/pkg/A.java

package pkg; // first non-comment statement

public class A {
    // public: other packages may call this method after importing pkg.A
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

src/pkg/B.java

```
package pkg;

public class B {
    // public: visible everywhere
    public static int twice(int x) {
        return 2 * x;
    }

    // no modifier = package-private: visible only inside pkg
    static String secret() {
        return "pkg only";
    }
}
```

src/pkg/Tester.java

```
package pkg;

public class Tester {
    // The class you run must contain a main method like this one.
    public static void main(String[] args) {
        A.greet(); // same package: no import needed
        System.out.println(B.twice(21)); // prints 42
        System.out.println(B.secret()); // legal here: same package
    }
}
```

Imports and visibility

Choose specific, wildcard, or fully qualified

```
import java.util.Scanner; // one class
import java.io.*;         // every public class directly in java.io
import pkg.A;             // a user-defined class

// Without an import you may write the fully qualified name instead:
pkg.A.greet();
```

- `java.lang` is imported automatically. Classes in the *same* package never need an import.
- A wildcard such as `java.io.*` imports the classes directly in that package, not classes in its sub-packages.
- An import only shortens names. It does not grant access to members that are not visible.

MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS IN ANOTHER PACKAGE	ANY OTHER CLASS
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
(none) · package-private	Yes	Yes	No	No
private	Yes	No	No	No

src/app/Main.java · uses another package

```
package app;

import pkg.A;    // one class from another package
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5));    // prints 10
        // B.secret();    // compile error: secret() is not public
    }
}
```

Compile and run from the project folder

Commands

```
javac -d classes -sourcepath src src/pkg/*.java
java -classpath classes pkg.Tester

javac -d classes -sourcepath src src/app/Main.java
java -classpath classes app.Main
```

1

`-sourcepath src` tells the compiler where source files live.

2

`-d classes` tells it where to create the package folders and `.class` files.

3

`-classpath classes` tells Java where compiled packages begin.

4

`pkg.Tester` uses the fully qualified class name: package plus class. It must contain `main`.

5

Forward slashes in the paths also work on Windows command prompts.

Build a package from scratch

1 Create `src/pkg` and `classes`.

2 Place package `pkg`; first in each source file.

3 Save source files inside `src/pkg`.

4 Give one class a `main` method.

5 Compile with `-d classes`.

6 Run the fully qualified class name.

Debug checklist

- Package name and folder path must match.
- Java is case-sensitive.
- Run from the folder above `classes`.
- Do not include `.class` in the run command.
- The class you run needs `main`.

Reading the error messages

MESSAGE	USUAL CAUSE	FIX
Error: Could not find or load main class <code>pkg.Tester</code>	Running from the wrong folder, a wrong classpath, a misspelled name, or adding <code>.class</code> .	Run from the folder that contains <code>classes</code> ; use <code>-classpath classes pkg.Tester</code> with no extension.
Error: Main method not found in class <code>pkg.A</code>	The class has no <code>public static void main(String[] args)</code> .	Run a class that has <code>main</code> , such as <code>pkg.Tester</code> .
error: package <code>pkg</code> does not exist	Compiling <code>app.Main</code> without telling <code>javac</code> where <code>pkg</code> lives.	Add <code>-sourcepath src</code> (or <code>-classpath classes</code> if <code>pkg</code> is already compiled).

MESSAGE	USUAL CAUSE	FIX
error: class Wrong is public, should be declared in a file named Wrong.java	A public class name must match its file name.	Rename the file or the class.
error: secret() is not public in B; cannot be accessed from outside package	Calling a package-private member from another package.	Make it public , or call it only from inside pkg.
Compiles, but <code>java pkg.A</code> says it cannot find the class	The package statement does not match the folder, so the <code>.class</code> landed in a different folder.	Make the package statement, the source folder, and the run command agree.

Worked example · source to running class

STAGE	INPUT	WHAT JAVA RESOLVES	RESULT
Declare	<code>package pkg; in src/pkg/Tester.java</code>	The source folder path must equal the package name	<code>src/pkg/Tester.java</code>
Compile	<code>javac -d classes -sourcepath src src/pkg/*.java</code>	<code>-d classes</code> creates <code>classes/pkg/</code> ; one <code>.class</code> per class	<code>classes/pkg/A.class</code> , <code>B.class</code> , <code>Tester.class</code>
Run	<code>java -classpath classes pkg.Tester</code>	Classpath root <code>classes</code> + <code>pkg.Tester</code> → <code>classes/pkg/Tester.class</code> ; then <code>main</code> runs	Hello from pkg.A · 42 · pkg only
Cross-package	<code>import pkg.A; in src/app/Main.java</code>	<code>-sourcepath src</code> lets <code>javac</code> find <code>src/pkg/A.java</code> when compiling <code>app.Main</code>	<code>classes/app/Main.class</code> ; prints Hello from pkg.A · 10

● ● ● Console output of `java -classpath classes pkg.Tester`

```
Hello from pkg.A
42
pkg only
```

Fully annotated Java and commands

java

```
package pkg; // Identity pkg.A; file src/pkg/A.java.

public class A { // Public class name must match A.java.
    // Other packages may call this public method on this public class.
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

java

```
package pkg; // File src/pkg/B.java.

public class B {
    public static int twice(int x) { return 2 * x; }
    // No access modifier: only classes in package pkg may call secret.
    static String secret() { return "pkg only"; }
}
```

java

```
package pkg; // File src/pkg/Tester.java.

public class Tester {
    // This exact main signature supplies the program entry point.
    public static void main(String[] args) {
        A.greet(); // Same package: no import required.
        System.out.println(B.twice(21)); // Prints 42.
        System.out.println(B.secret()); // Legal: Tester and B share pkg.
    }
}
```

java

```
package app; // File src/app/Main.java.

import pkg.A; // Import a public class from another package.
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5)); // Prints 10.
        // B.secret(); // Would fail: imports do not grant package-private access.
    }
}
```

Commands

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java
src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

Imports and access: different responsibilities

MEMBER MODIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS OUTSIDE PACKAGE	UNRELATED OUTSIDE CLASS
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes, with subclass access restrictions	No
none (package-private)	Yes	Yes	No	No
private	Yes	No	No	No

Protected access from a different package is through inheritance and is subject to receiver restrictions; it is not blanket permission to access any superclass object. A top-level class may be public or package-private, not protected/private.

Access comparison

```
import pkg.B;
// Legal in app.Main: B.twice(5)
// Illegal in app.Main: B.secret()
// Importing B does not alter secret() visibility.
```

Properties and practical cost

Packages organize names and collaboration; imports are compile-time name resolution conveniences. A wildcard is not a performance optimization. There is no useful sorting-style $O(n)$ claim to assign to an import declaration.

PROPERTY	PRACTICAL CONSEQUENCE
Namespace	Same simple type names may exist in different packages.
Encapsulation	Package-private members form an internal package API.
Maintainability	Related classes have predictable source locations.
Dependency lookup	Sourcepath/classpath roots must locate the required types.
Case sensitivity	pkg.A and pkg.a name different types.

Five worked package cases

1. One-level package

Givenpackage pkg; public class A

→

Resolution / resultsrc/pkg/A.java →
classes/pkg/A.class

Identity is pkg.A.

STEP	EVIDENCE	CONCLUSION
1	package pkg; public class A	Identify package/type/root
2	src/pkg/A.java → classes/pkg/A.class	Identity is pkg.A.

2. Nested package

Givenpackage ub.cs.tools; public class Helper

→

Resolution /
resultsrc/ub/cs/tools/Helper.java →
classes/ub/cs/tools/Helper.class

Identity is ub.cs.tools.Helper; roots are excluded.

STEP	EVIDENCE	CONCLUSION
1	package ub.cs.tools; public class Helper	Identify package/type/root
2	src/ub/cs/tools/Helper.java → classes/ub/cs/tools/Helper.class	Identity is ub.cs.tools.Helper; roots are excluded.

3. Same-package call

Givenpkg.Tester calls B.secret()

→

Resolution / resultNo import; package-private access allowed.

Both classes belong to pkg.

STEP	EVIDENCE	CONCLUSION
1	pkg.Tester calls B.secret()	Identify package/type/root
2	No import; package-private access allowed.	Both classes belong to pkg.

4. Cross-package call

Given `app.Main imports pkg.B`

→

Resolution / result `B.twice(5) → 10;`
`B.secret()` is inaccessible.

Import shortens the class name but does not grant member access.

STEP	EVIDENCE	CONCLUSION
1	<code>app.Main imports pkg.B</code>	Identify package/type/root
2	<code>B.twice(5) → 10; B.secret()</code> is inaccessible.	Import shortens the class name but does not grant member access.

5. Runtime root

Given `java -classpath classes app.Main`

→

Resolution / result `classes/app/Main.class → main → output`

The root is `classes`, not `classes/app`.

STEP	EVIDENCE	CONCLUSION
1	<code>java -classpath classes app.Main</code>	Identify package/type/root
2	<code>classes/app/Main.class → main → output</code>	The root is <code>classes</code> , not <code>classes/app</code> .

Common errors and exact repairs

MESSAGE / SYMPTOM	LIKELY CAUSE	REPAIR
cannot find symbol: Scanner	Simple name is not in scope.	Add <code>import java.util.Scanner</code> or use the fully qualified name.

MESSAGE / SYMPTOM	LIKELY CAUSE	REPAIR
reference to Date is ambiguous	Two wildcard-imported packages contain Date.	Use a specific import or fully qualified name.
package pkg does not exist	Compiler cannot locate sources/compiled dependencies.	Use -sourcepath src or -classpath classes as appropriate.
class A is public, should be declared in A.java	Public class and filename disagree.	Use A.java with public class A.
Could not find or load main class pkg.Tester	Wrong classpath root/name or missing compiled file.	Use classes as root and pkg.Tester without .class.
Main method not found in class pkg.A	That helper is not an entry-point class.	Run pkg.Tester or app.Main.
secret() is not public in B	Cross-package access to a package-private member.	Use a public API; an import cannot grant visibility.

Exam-style practice with worked answers

▼ A. Place Helper in ub.cs.tools. Give declaration, source path, output path and binary name. [4]

package ub.cs.tools; / src/ub/cs/tools/Helper.java / classes/ub/cs/tools/Helper.class / ub.cs.tools.Helper.
The src and classes roots are not package segments.

▼ B. Explain import java.util.* and whether it includes subpackages. [3]

It allows accessible types directly in java.util to be named simply. It does not import types from subpackages, copy code or install dependencies.

▼ **C. Why can `pkg.Tester` call `B.secret` while `app.Main` cannot? [3]**

`secret` has no access modifier, so it is package-private. `Tester` shares `pkg`. `Main` belongs to `app`; importing `B` does not grant member visibility.

▼ **D. Repair `java -classpath classes/pkg Tester.class`. [3]**

Use `java -classpath classes pkg.Tester`. The classpath names the root above package folders; the launch argument is a dotted binary name without an extension.

▼ **E. Compile and run the four classes in this chapter. State expected output. [6]**

Commands

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java
src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

`pkg.Tester` prints Hello from `pkg.A`, 42, `pkg` only. `app.Main` prints Hello from `pkg.A`, 10.

▼ **F. `Scanner` is unresolved even though the JDK is installed. What are two fixes? [2]**

Add `import java.util.Scanner`; or use `java.util.Scanner` at every relevant type/constructor use. Both name the same class.

Recommended implementation sequence

Complete the lab in dependency order so each piece is testable before the next one uses it. Step 1 is only folder set-up; steps 2–4 are written out in the marks application; steps 5–8 in the staff and files section.

1

Create folders and package. Make `src/lab05` and `classes`; put package `lab05`; at the top of each source file.

2

Implement merge. Test it on two already sorted halves before adding recursion.

3

Implement mergeSort. Use the base case, midpoint, left call, right call, then merge.

4

Build the marks application. Generate 30 marks from 0 to 100 with a `while` loop, print before and after, and repeat until the user stops.

5

Draw the UML first. Put shared fields and methods in `Doctor`; connect `GP` and `Surgeon` with inheritance arrows.

6

Implement and test each class. Test constructors, `tagName()`, and `calculatePay()` before file reading.

7

Read data.txt. Use field count to decide whether each record represents a `Doctor`, `GP`, or `Surgeon`.

8

Write the three output files. Route subclasses first, close resources, then inspect every output file.

Final check: compile from the folder above `src` and `classes`, then run with `java -classpath classes lab05.ClassName`.

The marks application: merge sort, random data, and a repeat loop

Both files live in `src/lab05/` and start with `package lab05;`. `MergeSort` holds the two methods; `MarksApp` is the class you run.

```
src/lab05/MergeSort.java

package lab05;

import java.util.Arrays;

public class MergeSort {
    public static void mergeSort(int[] arr, int start, int end) {
        // A zero- or one-value range is already sorted.
        if (start < end) {
            int mid = start + (end - start) / 2;
            mergeSort(arr, start, mid);    // sort the left half
            mergeSort(arr, mid + 1, end);  // sort the right half
            merge(arr, start, mid, end);    // combine the two sorted halves
        }
    }

    public static void merge(int[] arr, int s, int m, int e) {
        // Copy both halves; the right half starts at m + 1.
        int[] left = Arrays.copyOfRange(arr, s, m + 1);
        int[] right = Arrays.copyOfRange(arr, m + 1, e + 1);
        int i = 0, j = 0, cur = s;

        // Take the smaller front value each time (≤ keeps the merge stable).
        while (i < left.length && j < right.length)
            arr[cur++] = left[i] ≤ right[j] ? left[i++] : right[j++];

        // Exactly one of these loops copies the leftovers.
        while (i < left.length) arr[cur++] = left[i++];
        while (j < right.length) arr[cur++] = right[j++];
    }
}
```

```
package lab05;

import java.util.Arrays;
import java.util.Random;
import java.util.Scanner;

public class MarksApp {
    public static void main(String[] args) {
        Random random = new Random();
        Scanner keyboard = new Scanner(System.in); // create ONCE, outside the loop
        String again = "y";                        // lets the first pass run

        while (again.equalsIgnoreCase("y")) {
            int[] marks = new int[30];

            // Fill every cell with a random mark from 0 to 100 inclusive.
            int i = 0;
            while (i < marks.length) {
                marks[i] = random.nextInt(101); // upper bound is exclusive
                i++;
            }

            System.out.println("Before: " + Arrays.toString(marks));
            MergeSort.mergeSort(marks, 0, marks.length - 1);
            System.out.println("After:  " + Arrays.toString(marks));

            System.out.print("Run again? (y/n) ");
            again = keyboard.nextLine().trim();
        }
        keyboard.close();
    }
}
```

Compile and run

```
javac -d classes -sourcepath src src/lab05/*.java
java -classpath classes lab05.MarksApp
```

Each run prints a **Before**: line of 30 random marks, an **After**: line with the same 30 values in ascending order, and the prompt **Run again? (y/n)**. Answering **y** repeats with fresh random marks; anything else ends the loop.

DETAIL

`random.nextInt(101)`

WHY IT IS WRITTEN THAT WAY

The bound is *exclusive*, so `nextInt(101)` gives 0 to 100 inclusive. `nextInt(100)` could never produce 100.

DETAIL

WHY IT IS WRITTEN THAT WAY

```
int i = 0; while (i < marks.length) { ...
i++; }
```

The lab asks for a `while` loop. The three parts of a `for` loop are still there: start, condition, update.

```
String again = "y"
```

Starting at `y` lets the loop body run once before the user is asked anything.

```
Scanner keyboard created once
```

Closing a `Scanner` on `System.in` closes the keyboard stream. Create it before the loop and close it after.

```
MergeSort.mergeSort(marks, 0,
marks.length - 1)
```

Both bounds are *inclusive* indexes, so the last index is `length - 1`.

▼ Quick check: what would go wrong if `int[] marks = new int[30];` were placed before the `while` (again...) loop?

Nothing would visibly break. The fill loop sits inside the `while`, so every run still overwrites all 30 cells with new random marks, prints a fresh **Before:** line, and sorts it. Declaring the array inside the loop is still better: each run starts from a brand-new array, no values can leak from the previous run, and `marks` exists only where it is used.

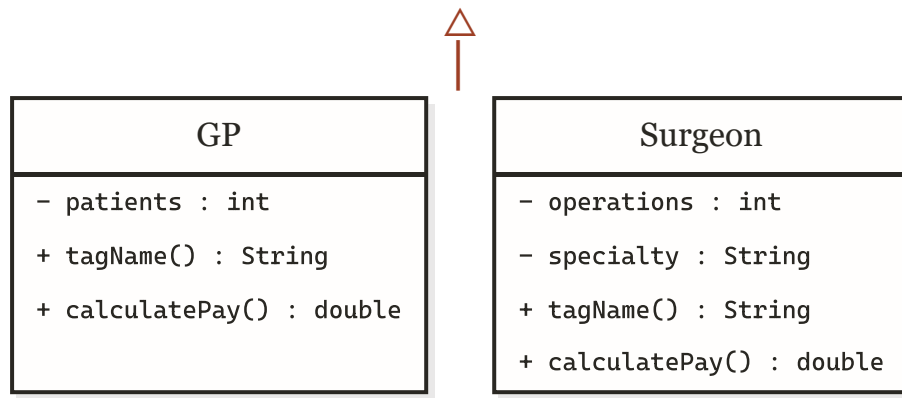
Lab05 / Steps 5–8

Inheritance, record routing, and three output files

Read this first. The class names and the sequence come from the lab outline, but this guide does not contain the lab handout. The field names, pay formulas, and field counts below are **placeholders that demonstrate the pattern**. Replace them with the exact specification you were given.

The pattern has four ideas: a shared superclass, subclasses that extend it, an array typed as the superclass that holds all three kinds, and routing each record to the right file.

Doctor
<pre># name : String # basePay : double + tagName() : String + calculatePay() : double</pre>



The hollow triangle points to the superclass. # protected, - private, + public.

src/lab05/Doctor.java

```

package lab05;

public class Doctor {
    protected String name;      // protected: visible to subclasses
    protected double basePay;

    public Doctor(String name, double basePay) {
        this.name = name;
        this.basePay = basePay;
    }

    public String tagName() { return "Dr " + name; }
    public double calculatePay() { return basePay; }
}
  
```

src/lab05/GP.java

```

package lab05;

public class GP extends Doctor {      // a GP IS-A Doctor
    private int patients;

    public GP(String name, double basePay, int patients) {
        super(name, basePay);        // must be the first statement
        this.patients = patients;
    }

    @Override public String tagName() { return "GP " + name; }
    @Override public double calculatePay() {
        return super.calculatePay() + patients * 2.0; // placeholder formula
    }
}
  
```

```

package lab05;

public class Surgeon extends Doctor {
    private int operations;
    private String specialty;

    public Surgeon(String name, double basePay, int operations, String specialty) {
        super(name, basePay);
        this.operations = operations;
        this.specialty = specialty;
    }

    @Override public String tagName() { return "Surgeon " + name; }
    @Override public double calculatePay() {
        return super.calculatePay() + operations * 50.0; // placeholder formula
    }
}

```

IDEA	IN THE CODE	WHY IT MATTERS
<code>extends</code>	GP <code>extends</code> <code>Doctor</code>	A GP is a Doctor, so it inherits <code>name</code> , <code>basePay</code> , and every public method.
<code>super(...)</code>	First line of each subclass constructor	Runs <code>Doctor</code> 's constructor to set the shared fields.
<code>@Override</code>	<code>tagName()</code> , <code>calculatePay()</code>	A subclass supplies its own version. <code>super.calculatePay()</code> reuses the parent's.
Superclass array	<code>Doctor[] staff</code>	A <code>Doctor</code> cell may hold a <code>Doctor</code> , <code>GP</code> , or <code>Surgeon</code> object. The call <code>staff[i].calculatePay()</code> runs the version belonging to the real object.

Read the records, then route them

src/lab05/StaffReport.java

```
package lab05;

import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintWriter;
import java.util.Scanner;

public class StaffReport {
    public static void main(String[] args) throws FileNotFoundException {
        Doctor[] staff = new Doctor[20]; // capacity; holds Doctor, GP and Surgeon
        int count = 0; // logical size

        try (Scanner in = new Scanner(new File("data.txt"))) {
            while (in.hasNextLine() && count < staff.length) {
                Doctor d = parse(in.nextLine());
                if (d != null) staff[count++] = d; // skip records with an unrecognised field count
            }
        }

        // All three writers are closed automatically, which flushes their output.
        try (PrintWriter doctors = new PrintWriter("doctors.txt");
            PrintWriter gps = new PrintWriter("gps.txt");
            PrintWriter surgeons = new PrintWriter("surgeons.txt")) {
            for (int i = 0; i < count; i++) {
                Doctor d = staff[i];
                // Test subclasses first: every GP and Surgeon is also a Doctor.
                PrintWriter target = (d instanceof Surgeon) ? surgeons
                    : (d instanceof GP) ? gps
                    : doctors;
                target.println(d.tagName() + " " + d.calculatePay());
            }
        }

        // The number of fields on the line decides which class to build.
        static Doctor parse(String line) {
            String[] f = line.trim().split("\\s+");
            switch (f.length) {
                case 2: return new Doctor(f[0], Double.parseDouble(f[1]));
                case 3: return new GP(f[0], Double.parseDouble(f[1]), Integer.parseInt(f[2]));
                case 4: return new Surgeon(f[0], Double.parseDouble(f[1]),
                    Integer.parseInt(f[2]), f[3]);
                default: return null; // wrong field count
            }
        }
    }
}
```

With this data.txt:

data.txt

```
Ann 5000
Bo 5000 40
Cy 5000 3 cardiac
broken
Dee 4000
```

LINE	FIELDS	OBJECT BUILT	WRITTEN TO
Ann 5000	2	Doctor	doctors.txt: Dr Ann 5000.0
Bo 5000 40	3	GP	gps.txt: GP Bo 5000.0
Cy 5000 3 cardiac	4	Surgeon	surgeons.txt: Surgeon Cy 5150.0
broken	1	none (null)	skipped
Dee 4000	2	Doctor	doctors.txt: Dr Dee 4000.0

Build it in this order

- 1 Draw the UML and decide what is shared.
- 2 Write and test **Doctor**.
- 3 Write **GP** and **Surgeon**; test each constructor and method.
- 4 Parse one line into one object.
- 5 Loop over the file, storing into the array with a **count**.
- 6 Open three writers and route each object.
- 7 Open every output file and check it.

Debug checklist

- Test **Surgeon** and **GP** before **Doctor** in the routing chain. A **Doctor** test first would match all three kinds.
- **super(...)** must be the first statement of a subclass constructor.
- Close the writers (try-with-resources does it) or buffered output may never reach the file.
- Skip a record whose field count is not recognised rather than crashing.

Package and lab questions

▼ Why is `classes` the classpath rather than `classes/Lab05`?

The classpath names the root above the package hierarchy. Java then uses `Lab05.ClassName` to find

`lab05/ClassName.class` below that root.

▼ **Where must `package lab05;` appear?**

It must be the first non-comment statement in the source file.

▼ **Why does `java -classpath classes pkg.A` fail even though `A.class` exists?**

Class A only defines `greet()`. To start a program Java needs `public static void main(String[] args)` in the named class. Run `pkg.Tester`, which contains it.

▼ **What is the difference between an `import` and a fully qualified name?**

Both reach the same class. An import lets you write the short name A for the rest of the file; the fully qualified name `pkg.A` spells out the package each time. Neither bypasses visibility rules.

▼ **Can a `Doctor[]` array hold a GP?**

Yes. A GP is a Doctor, so a Doctor reference may refer to it. The array stores references, so the cell holds a reference to the GP object and `calculatePay()` runs the GP version.

Packages in one sentence

The declared package, source folders, compiled folders, imports, classpath root, and fully qualified class name must all describe the same hierarchy, and the class you run must contain `main`.

Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What does an import do?

- ☐ A. Allows an accessible type to be written by a shorter name
- ☐ B. Installs a library
- ☐ C. Changes private members to public

2. Which package is automatically imported?

- ☐ A. java.util
- ☐ B. java.io
- ☐ C. java.lang

3. Does import java.util.* include java.util.concurrent types?

- ☐ A. Only if they are public
- ☐ B. No
- ☐ C. Yes, recursively

4. What comes first in a packaged source file, after comments?

- ☐ A. package declaration
- ☐ B. import declarations
- ☐ C. public class declaration

5. What is the identity of public class Helper declared in ub.cs.tools?

- ☐ A. src.ub.cs.tools.Helper
- ☐ B. Helper.java
- ☐ C. ub.cs.tools.Helper

6. Correct source path below source root src?

- ☐ A. ub/cs/src/Helper.java
- ☐ B. src/ub/cs/tools/Helper.java
- ☐ C. src/ub.cs.tools/Helper.class

7. What does javac -d classes select?

- ☐ A. The output root for compiled classes
- ☐ B. The package name
- ☐ C. The class containing main

8. How do you launch classes/pkg/Tester.class from the project root?

- ☐ A. java -classpath classes/pkg Tester.class
- ☐ B. javac pkg.Tester
- ☐ C. java -classpath classes pkg.Tester

9. Can app.Main use a package-private method in pkg.B after importing B?

- ☐ A. Yes, if it is static
- ☐ B. No
- ☐ C. Yes, imports grant access

10. Must two classes in the same package import one another?

- ☐ A. No
- ☐ B. Yes, always
- ☐ C. Only if both are public

11. How can a conflict between `java.util.Date` and `java.sql.Date` be resolved?

- ☐ A. Add more wildcard imports
- ☐ B. Rename `java.lang`
- ☐ C. Use a fully qualified name or the appropriate specific import

12. What does a wrong classpath root usually prevent?

- ☐ A. Allocating an array
- ☐ B. Finding the compiled main class
- ☐ C. Parsing the package declaration in source

▼ Answer key and explanations

1. **Allows an accessible type to be written by a shorter name.** Imports affect compile-time name resolution, not installation or access control.
2. **java.lang.** String, System and Math are java.lang types and can use simple names automatically.
3. **No.** A wildcard import covers accessible types directly in that package, not its subpackages.
4. **package declaration.** The normal source order is package, imports, then top-level type declarations.
5. **ub.cs.tools.Helper.** Source roots and filename extensions are not part of the fully qualified class name.
6. **src/ub/cs/tools/Helper.java.** Package segments map to directories below the root; source uses the .java extension.
7. **The output root for compiled classes.** The compiler creates package directories beneath the output root.
8. **java -classpath classes pkg.Tester.** The classpath ends above pkg; the launch argument is a dotted name with no extension.
9. **No.** Importing a class never widens its members' access.
10. **No.** Accessible types in the same package can be named directly.
11. **Use a fully qualified name or the appropriate specific import.** Qualification tells the compiler exactly which Date you mean.
12. **Finding the compiled main class.** Runtime lookup combines the root with the dotted binary name to locate the class file.